The target Machine

* Familiarity with the target machine and its instruction set is a prerequisite for designing a good code generator.

* Consider the target computer is a byte-addressable machine with four bytes to a word and n general purpose registers $R_0, R_1, \ldots R_{n-1}$. It has two address instructions of the form OP Source, destination.

→ MOV R0,M, stores the contents of register R0 into memory location M.

Instruction costs

* This cost corresponds to the length in words of the instruction. Instructions involving registers have cost one, while those with a memory location or literal in them have cost two.

* MOV R0,R1 copies the contents of register R0 into register R1. This instruction has cost one, since it occupies only one word of memory.

MOV	R0	R1
2 byte	1 byte	1 byte

* MOV R5,M copies the contents of register R5 into memory location M. This instruction has

cost two, since the address of memory location M is in the word following the instruction.

① MOV b, R0 2 words
ADD C, R0 2 words cost = 6.
MOV R0, a 2 words

② MOV b, a 3 words cost = 6
ADD c, a 3 words

③ MOV R1, R0 1 word cost = 2.
ADD R2, R0 1 word

④ ADD R2, R1 1 word cost = 3
MOV R1, a 2 words

* In order to generate good code for the machine, utilize ~~A simp~~ its addressing capabilities efficiently.

A Simple Code Generator

* It generates target code for a sequence of 3-address statement.
* Consider that the computed results can be left in registers as long as possible, storing them only
(a) if their register is needed for another computation.

(b) Just before a procedure call or jump.
This implies that everything must be stored just before the end of a basic block.

Register and Address descriptors

* The code generation algorithm uses descriptors to keep track of register contents and addresses for names.
* A register descriptor keeps track of what is currently in each register. It is consulted whenever a new register is needed.
* Initially the reg. descriptor shows that all regs. are empty are assigned across blocks. As the code generation for the block progresses, each reg. will hold the value of zero or more names at any given time.
* An address descriptor keeps track of the location or locations where the current value of the name can be found at run time. The location might be a reg; a stack location or a memory address.

A Code generation algorithm

* The code generation algorithm takes as input a sequence of three address statements constituting a basic block.
* For each three-address statement of the form $\boxed{x = y \text{ op } z}$, we perform the following actions.

① Invoke a function getreg to determine location L where the result of the computation y op z should be stored. L will usually be a reg, but it could also be a memory location.

② Consult the address descriptor for y to determine y, (one of) the current location of y. Prefer the reg. for y if the value of y is currently both in memory and a reg. If the value of y is not already in L, generate the instruction, MOV y, L to place a copy of y in L.

③ Generate the instruction OP z, L. Again prefer a reg. to a memory location if z is in both. Update the address descriptor of

x to indicate that x is in location L.
If L is a register, update its descriptor to indicate that it contains the value of x, and remove x from all other register descriptors.

(4) If the current value of y and/or z have no next uses, are not live on exit from the block and are in regs, alter the register descriptor to indicate that, after execution of x = y op z, those regs no longer will contain y and/or z respectively.

The function getreg.

The function getreg returns the location L to hold the value of x for the assignment x = y op z.

* The assignment d = (a-b)+(a-c)+(a-c) might be translated in to the following 3-address code sequence

```
t = a-b
u = a-c
v = t+u
d = v+u.
```

Statements	Code generated	Register descriptor	Address Descriptor
t = a - b	MOV a, R0 SUB b, R0	R0 contains t	t in R0
u = a - c	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1
v = t + u	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R0
d = v + u	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory.

Fig. Code sequence.

- The cost of the code generated is 12.
- Could reduce this to 11 by generating MOV R0, R1 immediately after the first instruction and removing the instruction MOV a, R1, but to do so requires a more sophisticated code generation algorithm.
- The reason for the savings is that it is cheaper to load R1 from R0 than from memory.